

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  dt = 0.1
5  t_max = 10.0
6  steps = int(t_max / dt)
7
8  # State: [position, velocity]
9  A = np.array([[1, dt],
10 |             | [0, 1]])
11  B = np.zeros((2, 1))
12  C = np.array([[1, 0]])
13
14  # Q: Process Noise (Model Uncertainty)
15  q_var_pos = 0.1
16  q_var_vel = 0.1
17  Q = np.array([[q_var_pos**2, 0],
18 |             | [0, q_var_vel**2]])
19
20  # R: Measurement Noise (Sensor Uncertainty)
21  r_var = 1.0
22  R = np.array([[r_var**2]])
23
24  # True State Initialization
25  true_x = np.array([[0.0], [5.0]])
26
27  # Kalman Filter State Initialization
28  x_hat = np.array([[0.0], [0.0]])
29  P = np.array([[100.0, 0.0],
30 |             | [0.0, 100.0]])
31
32  # Storage for results
33  true_states = [true_x.copy()]
34  measurements = []
35  estimated_states = [x_hat.copy()]

```

```

36  #Simulation and Kalman Filter Loop
37  for k in range(steps):
38      process_noise = np.array([[np.random.normal(0, q_var_pos)],
39 |                             | [np.random.normal(0, q_var_vel)]])
40      true_x = A @ true_x + process_noise
41      true_states.append(true_x.copy())
42
43      measurement_noise = np.array([[np.random.normal(0, r_var)]])
44      z_k = C @ true_x + measurement_noise
45      measurements.append(z_k.copy())
46
47      #PREDICT (Time Update)
48      x_hat_minus = A @ x_hat + B @ np.zeros((1, 1))
49      P_minus = A @ P @ A.T + Q
50
51      #UPDATE (Measurement Update)
52      S = C @ P_minus @ C.T + R
53      K = P_minus @ C.T @ np.linalg.inv(S)
54
55      y = z_k - C @ x_hat_minus
56      x_hat = x_hat_minus + K @ y
57
58      P = (np.eye(2) - K @ C) @ P_minus
59
60      # Store estimated state
61      estimated_states.append(x_hat.copy())
62
63  true_states = np.array(true_states)
64  measurements = np.array(measurements)
65  estimated_states = np.array(estimated_states)
66
67  plt.figure(figsize=(12, 8))
68
69  # Position Plot
70  plt.subplot(2, 1, 1)
71  plt.plot(np.linspace(0, t_max, steps + 1), true_states[:, 0], 'r--', Label='True Position')
72  plt.plot(np.linspace(0, t_max, steps + 1), measurements[:, 0], 'kx', alpha=0.5, Label='Noisy Measurements')
73  plt.plot(np.linspace(0, t_max, steps + 1), estimated_states[:, 0], 'b-', Label='KF Estimated Position')
74  plt.title('Position Estimation (1D Motion)')
75  plt.ylabel('Position (m)')
76  plt.legend()
77  plt.grid(True)
78
79  # Velocity Plot
80  plt.subplot(2, 1, 2)
81  plt.plot(np.linspace(0, t_max, steps + 1), true_states[:, 1], 'r--', Label='True Velocity')
82  plt.plot(np.linspace(0, t_max, steps + 1), estimated_states[:, 1], 'b-', Label='KF Estimated Velocity')
83  plt.title('Velocity Estimation (1D Motion)')
84  plt.xlabel('Time (s)')
85  plt.ylabel('Velocity (m/s)')
86  plt.legend()
87  plt.grid(True)
88
89  plt.tight_layout()
90  plt.show()

```

